



Contents lists available at ScienceDirect

Journal of Engineering Research

journal homepage: www.journals.elsevier.com/journal-of-engineering-research

Full length article

SMS spam detection via a systematic machine learning pipeline: Feature engineering, hyperparameter optimizing, scaling, and ensemble learning

Tuğçe Şenol^b, Hidayet Takcı^b, Abdulkadir Şeker^{a,b,*}^a Kyrgyz-Turkish Manas University, Department of Computer Engineering, Bishkek, Kyrgyzstan^b Sivas Cumhuriyet University, Department of Computer Engineering, Sivas, Turkey

ARTICLE INFO

Keywords:

Short message service spam detection
Machine learning
Ensemble learning
Hyperparameter optimization
Feature engineering
Text preprocessing

ABSTRACT

SMS spam detection remains a critical challenge as spammers continuously adapt their tactics to evade automated filters. This study proposes a systematic machine learning pipeline for SMS spam detection and conducts a comprehensive comparative analysis across multiple feature extraction methods, feature selection strategies, scaling approaches, and ensemble learning algorithms within a single reproducible framework. Experiments are performed on the UCI SMS Spam Collection v.1 dataset using eight classifiers (LR, DT, KNN, SVM, NC, GB, XGB, RF) combined with Bag-of-Words, TF-IDF, and Word2Vec feature extraction methods. Five feature selection methods (Chi-square, Mutual Information, LASSO, Random Forest Importance, and Recursive Feature Elimination), eight scaling strategies, and class imbalance handling via SMOTE and random undersampling are systematically evaluated. A standardized preprocessing pipeline incorporating lowercasing, URL normalization, digit normalization, and special character removal is applied prior to feature extraction. Results demonstrate that BoW consistently outperforms TF-IDF and Word2Vec across most classifiers. The best-performing configuration, BoW combined with QuantileTransformer scaling and Logistic Regression, achieves an accuracy of 0.989, precision of 0.995, recall of 0.920, F1-score of 0.956, and ROC-AUC of 0.990, with only 18 false negatives and 1 false positive on a test set of 1672 messages. Feature selection improves over baseline performance, with LASSO combined with SVM or XGB yielding the highest F1-scores on BoW features. Class imbalance experiments reveal that neither SMOTE nor random undersampling improves upon the unbalanced baseline, confirming that BoW features inherently tolerate the natural class distribution. XGBoost achieves the best ensemble results with the most favorable accuracy-efficiency trade-off. False negative analysis identifies personal/chat-mimicking spam and humor-format spam as the primary challenge categories for token-frequency methods. Training completes in under 2 s on a standard CPU without GPU dependency, confirming that competitive classification performance against deep learning approaches is attainable at substantially lower computational cost.

Introduction

The use of short message service (SMS) messages has increased significantly with the widespread adoption of mobile devices [1]. Individuals and organizations rely on SMS for communication, information sharing, two-factor authentication, and service notifications. Despite their widespread utility, SMS messages have become a primary vector for unsolicited and malicious content. The volume of misleading advertisements, phishing attempts, and fraudulent messages delivered via SMS continues to grow, posing serious threats to user security and communication quality. Automatically classifying SMS messages as spam or legitimate (ham) has therefore become a critical research

problem.

Traditional approaches to SMS spam detection include keyword-based, rule-based, whitelist/blacklist-based, and content-based methods [2–4]. Keyword-based methods label messages containing predefined spam terms such as “free, prize, or winner” as spam. Although simple and fast, these methods are prone to false positives and can be circumvented by minor textual modifications. Rule-based systems search for structural patterns associated with spam, while whitelist/blacklist approaches filter messages by sender identity. Content-based methods analyze statistical characteristics such as message length and capitalization rate. As spammers have evolved their strategies and adapted message structures to evade these filters, the

* Corresponding author at: Kyrgyz-Turkish Manas University, Department of Computer Engineering, Bishkek, Kyrgyzstan.

E-mail address: abdulkadir.seker@manas.edu.kg (A. Şeker).

<https://doi.org/10.1016/j.jer.2026.05.020>

Received 17 November 2025; Received in revised form 10 May 2026; Accepted 22 May 2026

Available online 25 May 2026

2307-1877/© 2026 The Authors. Published by Elsevier B.V. on behalf of Kuwait University. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

effectiveness of traditional methods has progressively diminished [5].

Modern approaches address the limitations of rule-based systems through data-driven learning. Machine learning (ML) methods automatically learn discriminative patterns from labeled examples, eliminating the need for manually crafted rules. More recently, deep learning (DL) models and large language models (LLMs) have demonstrated strong performance by capturing sequential dependencies and semantic representations in text [6]. Despite these advances, a comprehensive and reproducible comparison of diverse ML techniques spanning multiple feature extraction methods, feature selection strategies, scaling approaches, and ensemble methods within a single integrated framework remains lacking in the literature.

This study addresses this gap by proposing an integrated machine learning pipeline for SMS spam detection. The key contributions of this study are as follows:

- (1) Both BoW and TF-IDF feature extraction methods are systematically compared on the same dataset and classifier set.
- (2) Five feature selection methods Chi-square, Mutual Information, LASSO, Random Forest Importance, and Recursive Feature Elimination are evaluated and compared.
- (3) Eight scaling strategies are investigated to assess their effect on classification performance.
- (4) Eight classifiers, including five classical ML models and three ensemble methods, are comprehensively benchmarked within a single pipeline.
- (5) The effect of class imbalance is explicitly investigated through SMOTE and random undersampling experiments.
- (6) A text preprocessing pipeline incorporating lowercasing, URL normalization, digit normalization, and special character removal is applied and its effect on model performance is analyzed.

The remainder of this paper is organized as follows. 2 reviews related work. 3 describes the dataset. 4 describes the proposed model. 5 presents the experimental results. Sections 6–7 concludes the paper.

Related studies

SMS spam detection has been studied extensively across three main paradigms.

Machine learning-based approaches

The vulnerability of classical ML models to adversarial text modifications where spammers deliberately alter message structure to evade filters has also been documented, underscoring a key limitation of frequency-based approaches [5]. Machine learning methods have been the dominant paradigm in SMS spam detection over the past decade. SVM with parameter optimization and TF-IDF features were shown to achieve strong performance on the UCI SMS Spam Collection dataset, highlighting the critical role of feature engineering [7]. A comparative study across Naive Bayes, SVM, Decision Tree, and Random Forest confirmed that feature selection significantly improves classification accuracy across all evaluated models [8]. Baseline performance of classical classifiers on the same benchmark was established through systematic evaluation of Naive Bayes, Decision Tree, and Random Forest [9]. Beyond mainstream classifiers, TF-IDF combined with the Relevance Vector Machine demonstrated that less commonly used classifiers can achieve competitive results [10]. Random Forest paired with TF-IDF was shown to yield strong performance at low computational cost [11], while the H2O AutoML framework was leveraged for high-volume SMS spam detection, uncovering the potential of automated pipelines for large-scale data processing [12]. A recent comparison of multiple ML models including Logistic Regression, SVM, and ensemble methods confirmed that ensemble approaches consistently outperform individual

classifiers [13].

Topic modeling and semantic approaches

Several studies have explored semantic and topic-based representations. In a study that applied Latent Dirichlet Allocation (LDA) to extract hidden semantic features from SMS messages [14]. Another hybrid model combining probabilistic topic modeling with stacked denoising autoencoders [15]. These approaches require larger datasets to learn meaningful topic distributions, which limits their applicability on small benchmarks.

Deep learning-based approaches

Deep learning methods have gained significant traction in SMS spam detection due to their ability to learn feature representations directly from raw text. Convolutional Neural Networks were applied to SMS classification with custom feature engineering, demonstrating competitive performance on standard benchmarks [16]. Recurrent architectures “LSTM and GRU” have been particularly effective in capturing sequential dependencies in text, with both approaches achieving accuracy above 0.98 on the UCI dataset [17,18]. A CNN-based classification model evaluated on a locally collected Nigerian SMS dataset confirmed that deep learning generalizes across different linguistic contexts and data sources [19]. More recent work has explored combining TF-IDF vectorization with BiLSTM architectures, showing that frequency-based features can complement sequential models rather than replace them [20]. A hybrid GRU-CNN architecture proposed for Turkish and English SMS messages in 2024 demonstrated that multi-architecture fusion can further improve performance across diverse linguistic settings [21]. Most recently, a graph-augmented BERT model integrating co-occurrence, heterogeneous, and syntactic graph embeddings on the UCI benchmark, representing the current state-of-the-art among deep learning approaches [14,22].

LLM-based approaches

LLMs represent the most recent trend in SMS spam detection. Competitive performance was demonstrated using GPT-4, DeepSeek, LLaMA-2, and Mixtral, though at significantly higher inference cost compared to classical approaches [23]. RoBERTa combined with explainable AI techniques was proposed to increase transparency in LLM-based spam detection [24]. Despite these advances, a comparative study confirmed that TF-IDF paired with classical classifiers remains competitive with LLM-based approaches on standard benchmarks at substantially lower computational cost, motivating the continued use of classical ML pipelines as practical and reproducible alternatives [25].

Research gap and positioning of this study

As shown in Table 1, prior studies have generally focused on a single feature extraction method, a limited number of classifiers, or a single paradigm. This study fills these gaps by providing a comprehensive, reproducible comparison across feature extraction, feature selection, scaling, ensemble learning, and class imbalance handling within a single integrated pipeline.

Dataset

This study uses the UCI SMS Spam Collection v.1 dataset [26], consisting of 5574 labeled English SMS messages: 4823 (86.6%) ham and 747 (13.4%) spam. The model split 70/30 into training and test sets using stratified sampling.

The dataset exhibits significant class imbalance (~6.5:1), as shown in Fig. 1.

Spam messages show higher and more consistent word counts

Table 1

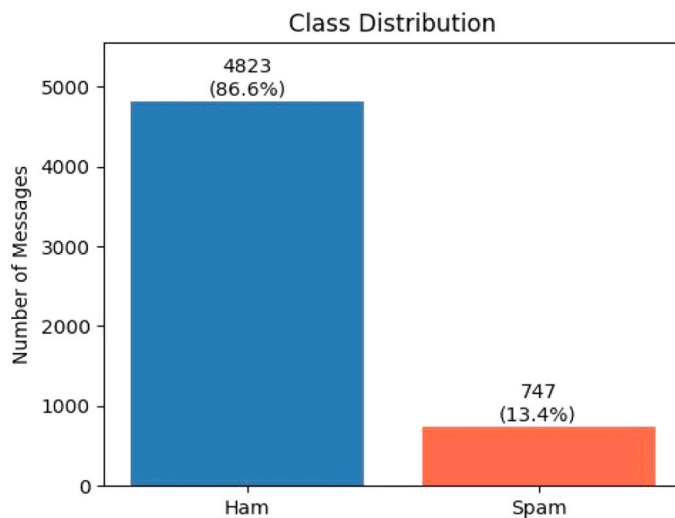
Summary of related work in SMS spam detection.

Study	Year	Method	Feature Extraction	Feature Selection	Ensemble
Tekerek [7]	2019	SVM	TF-IDF	No	No
Gupta et al. [8]	2018	NB, SVM, DT, RF	TF-IDF	Yes	RF
Sethi et al. [9]	2017	NB, DT, RF	BoW	No	No
Ma et al. [14]	2016	LDA	Topic vectors	No	No
Pudasaini et al. [10]	2023	RVM	TF-IDF	No	No
Sjarif et al. [11]	2019	RF	TF-IDF	No	RF
Suleiman et al. [12]	2017	H2O AutoML	TF-IDF	No	GB, RF
Popovac et al. [16]	2018	CNN	Embeddings	No	No
Roy et al. [17]	2020	LSTM+GRU	Embeddings	No	Yes
Abayomi-Alli et al. [19]	2022	CNN	Embeddings	No	No
Basumatary et al. [13]	2025	LR, SVM, RF, XGB	TF-IDF	No	RF
Goma et al. [20]	2024	LSTM, BiLSTM	TF-IDF	No	No
Altunay et al. [21]	2024	GRU+CNN hybrid	Embeddings	No	Yes
Salman et al. [23]	2025	GPT-4, LLaMA-2, DeepSeek	LLM embedding	No	Yes
Uddin et al. [24]	2024	RoBERTa + XAI	LLM embedding	No	No
Shen et. al [22]	2025	BERT-G3CN + Graph	BERT	No	No
This study	2026	LR, DT, KNN, SVM, NC, GB, XGB, RF	BoW + TF-IDF	Chi2, MI, LASSO, RFI, RFE	GB, XGB, RF

Table 2

Dataset information.

Property	Value
Total (original – preprocessed)	5574–5570
Ham - Spam	4822 (86.6%) - 747 (13.4%)
Train - Test	3899 (70%) - 1671 (30%)

**Fig. 1.** Class distribution of the dataset.

(median ~25) compared to ham (median ~11), as shown in Fig. 2.

Spam vocabulary is dominated by “free”, “call”, “claim”, “win”. Ham vocabulary reflects everyday conversation. The contrast justifies frequency-based feature extraction (Fig. 3).

The preprocessing pipeline: (1) lowercasing, (2) URL→ “url”, (3) digits→ “num”, (4) non-alphabetic removal, (5) whitespace normalization. Three empty messages removed after preprocessing.

Proposed model

Our study proposes a machine learning-based model for classifying SMS spam messages.

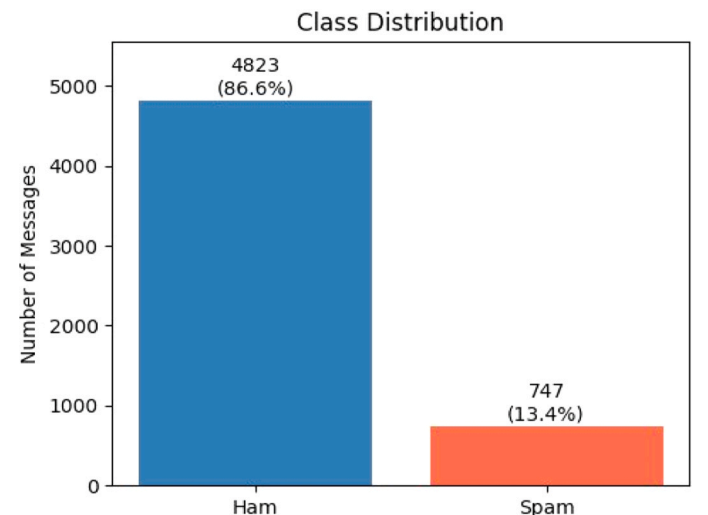
The study has a multi-stage integrated ML pipeline structured around four experimental setups:

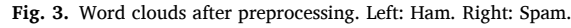
- (1) baseline classification
- (2) hyperparameter-optimized classification
- (3) feature selection
- (4) ensemble learning

The model architecture is shown in Fig. 4.

The proposed model is structured around four experimental setups for SMS spam detection, as illustrated in Fig. 4. Prior to all setups, a standardized preprocessing pipeline is applied to normalize the raw text, followed by feature extraction using Bag of Words (BoW), Term Frequency - Inverse Document Frequency (TF-IDF) or Word2Vec representations;

- (1) The first setup establishes baseline classification performance by applying five classical machine learning algorithms Logistic Regression (LR), Decision Tree (DT), K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Nearest Centroid (NC) directly to the extracted features. Additionally, the effect of class imbalance is investigated within this setup through SMOTE oversampling and random undersampling experiments.
- (2) The second setup applies GridSearchCV hyperparameter optimization with 5-fold stratified cross-validation to the same classifiers, aiming to identify the parameter configurations that maximize classification accuracy.
- (3) The third setup introduces feature selection as an intermediate step between feature extraction and classification, evaluating five selection methods “Chi-square (Chi²), Mutual Information (MI), LASSO, Random Forest Importance (RFI) and Recursive Feature Elimination (RFE)” to reduce dimensionality and retain the most

**Fig. 2.** Word count distribution by class.



The random forest importance provides an internal measure of feature importance based on how much each feature contributes to reducing impurity in the trees in the forest [32]. Features with higher

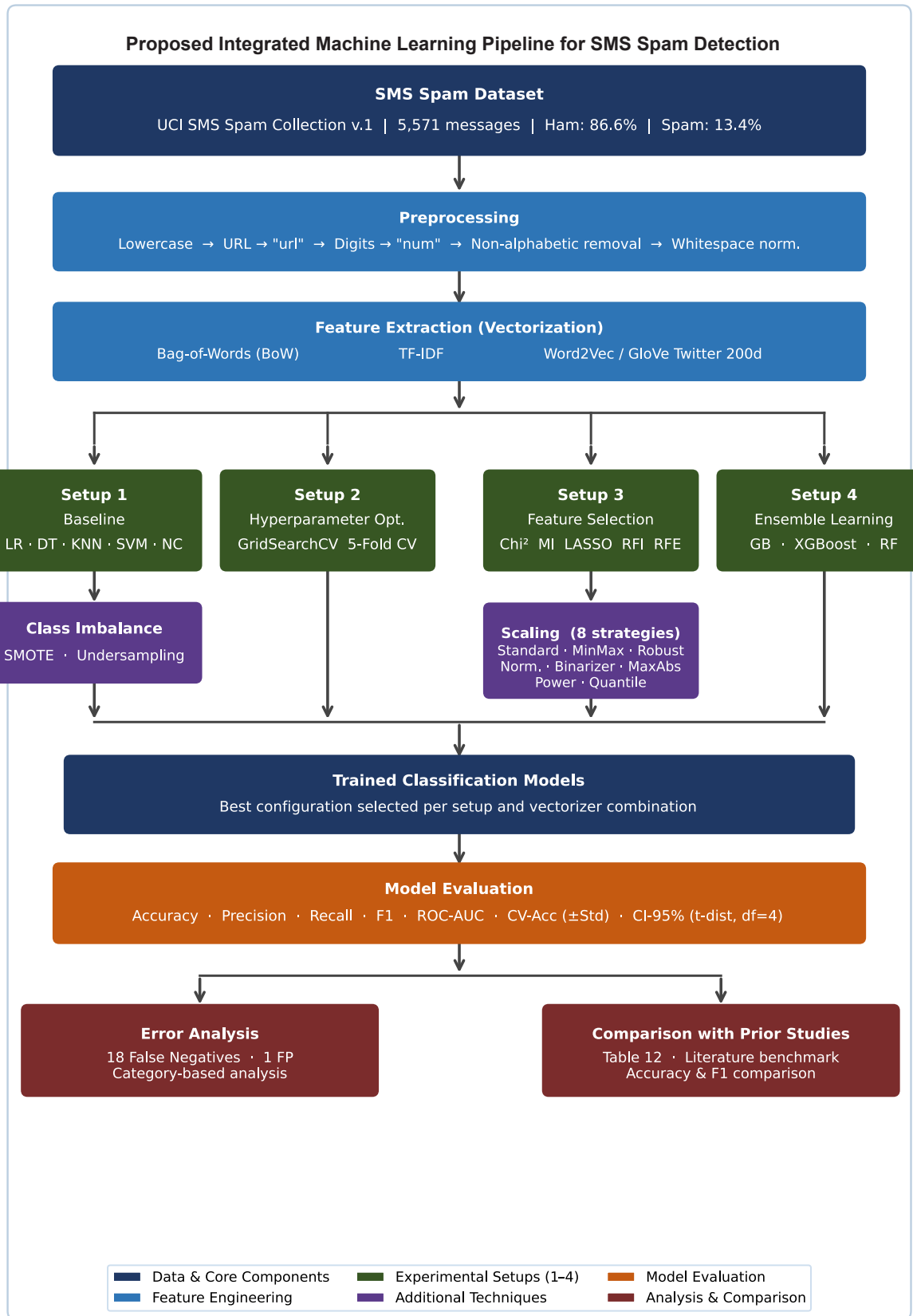


Fig. 4. The proposed pipeline for SMS spam detection.

importance scores are considered more relevant for classification.

The RFE is a wrapper-based feature selection method that recursively removes the least important features and trains a model on the remaining features [28]. The process continues until the desired number of features is reached. Feature importance is usually determined by the coefficients of a linear model or the feature importance scores of a tree-based model.

Scaling strategies

Scaling transforms feature values to a common range or distribution, which can significantly affect the performance of distance-sensitive and gradient-based classifiers such as KNN, SVM, and LR. Tree-based models such as DT, RF, and XGB are invariant to scaling and are unaffected by this step.

In this study, eight scaling strategies are evaluated to systematically assess their effect on classification performance across all vectorizer-classifier combinations. An important practical consideration arises when applying scaling to BoW and TF-IDF representations, which produce sparse matrices. Scaling methods that perform mean-centering such as StandardScaler, MinMaxScaler, and PowerTransformer convert sparse matrices to dense format, substantially increasing memory consumption. In contrast, MaxAbsScaler, Normalizer, and Binarizer operate directly on sparse matrices without centering, preserving sparsity. For Word2Vec representations, which produce dense matrices by design, all scaling strategies can be applied without this constraint (Table 3).

Ensemble learning algorithms

Ensemble learning is a technique that combines the predictions of multiple models to create a single, more accurate and robust model [33]. The GB, XGB, and RF algorithms were used for ensemble learning in this study. The GB is an ensemble learning algorithm that combines multiple decision trees to create a robust prediction model. The GB methods iteratively correct errors in previous models and improve overall accuracy. There are several popular varieties of it, such as the XGB, LightGBM, and CatBoost [34]. XGB is an efficient, scalable, and portable gradient boosting algorithm. Its superior performance has made it popular and widely used in many problems. RF is an ensemble learning algorithm in which each tree is trained with a random subset of data and features. The final prediction is made by combining the predictions of individual trees. RF is robust to overfitting and provides feature importance estimates.

Table 3

Scaling strategies evaluated in the study.

Scaler	Formula / Approach	Output Range	Sparse Matrix?	Best Use Case
StandardScaler	$z = (x - \mu) / \sigma$	$\mu=0, \sigma=1$	✗	Normal distribution assumed
MinMaxScaler	$(x - \min) / (\max - \min)$	$[0, 1]$	✗	Bounded range required
RobustScaler	$(x - \text{median}) / \text{IQR}$	Centered	✗	Outliers present
Normalizer	$x / \ x\ _2$ (row-wise)	Unit norm	✓	Text / cosine similarity
Binarizer	$x > 0 \rightarrow 1$, else $\rightarrow 0$	$\{0, 1\}$	✓	Presence/absence features
MaxAbsScaler	$x / \max(x)$	$[-1, 1]$	✓	Sparse matrices
PowerTransformer	Yeo-Johnson transform	Gaussian-like	✗	Skewed distributions
QuantileTransformer	Quantile mapping	Normal/Uniform	✗	Highly skewed + outliers

Hyperparameter optimization

In this study, hyperparameter optimization was performed using GridSearchCV. GridSearchCV is a frequently used method for improving the performance of machine learning models (Table 4). This method aims to determine the optimal set of parameters by trying all possible combinations within a given parameter range. Using a combination of possible parameters and cross-validation, GridSearchCV tests all parameter combinations and evaluates the model's accuracy.

Experimental setup

As seen in Table 5, All experiments use Python 3.10 with scikit-learn 1.3, XGBoost 1.7, and imbalanced-learn 0.11. Dataset split: 70/30 stratified (random_state=111). Vectorizers fit only on training set. Performance metrics: Accuracy, Precision, Recall, F1, ROC-AUC. Stability: 5-fold stratified CV (CV-Acc $\pm$ CV-Std). Confidence intervals computed using t-distribution with df= 4 (CI-95%).

Results

Class imbalance analysis

SMOTE and random undersampling are applied exclusively to the training set (Table 6). Neither improves upon the baseline. SMOTE causes dramatic KNN degradation (Acc: 0.966 $\rightarrow$ 0.536, F1:

Table 4

Hyperparameter optimizations of models.

Classifier	Hyperparameter	Search Grid
Logistic Regression	C	[0.01, 0.1, 1, 10, 100]
Decision Tree	max_depth, min_samples_split	[3,5,10,None] $\times$ [2,5,10]
KNN	n_neighbors, metric	[3,5,7,9,11] $\times$ [euclidean, manhattan]
SVM	C, kernel	[0.1,1,10,100] $\times$ [linear, rbf]
Gradient Boosting	n_estimators, lr, max_depth	[50,100,200] $\times$ [0.01,0.1,0.2] $\times$ [3,5]
XGBoost	n_estimators, lr, max_depth	[50,100,200] $\times$ [0.01,0.1,0.2] $\times$ [3,5,7]
Random Forest	n_estimators, max_depth, max_features	[50,100,200] $\times$ [None,10,20] $\times$ [sqrt,log2]

Table 5

Experimental setup specifications.

Component	Specification
Language/ Framework	Python 3.10, scikit-learn 1.3, XGBoost 1.7
Train/test split	70%/30%, stratified, random_state= 111
Cross-validation	5-fold StratifiedKfold, random_state= 42
CI formula	t-distribution, df= 4, $\alpha= 0.05$
Metrics	Accuracy, Precision, Recall, F1, ROC-AUC, CV-Acc, CV-Std, CI-95%
Feature selection k	100 hi2, MI, RFI, RFE); 144 auto (LASSO)

0.863→0.363). BoW features inherently tolerate the class imbalance through lexical separation. No resampling is applied in subsequent experiments.

Feature extraction analysis

Tables 7–9 presents baseline results for BoW, TF-IDF, and Word2Vec. SVM achieves highest F1 with BoW (0.940) and TF-IDF (0.934). KNN improves with TF-IDF (0.863→0.910); LR degrades (0.938→0.883). NC shows high ROC-AUC (0.9955) but lower accuracy, indicating unexploited discriminative potential. Word2Vec remains below BoW/TF-IDF for most classifiers.

Hyperparameter optimization analysis

GridSearchCV optimization consistently improved performance (Table 10). Most notable: SVM+TF-IDF (0.983→0.987); LR+TF-IDF (0.971→0.985). Bigram features (ngram_range=(1,2)) were selected for most classifiers, demonstrating the value of short phrase-level features.

Feature selection analysis

Feature selection experiments were conducted across five methods (Chi-square, Mutual Information, LASSO, Random Forest Importance, and Recursive Feature Elimination), eight classifiers (LR, DT, KNN, SVM, NC, GB, XGB, RF), and three vectorizers (BoW, TF-IDF,

Table 6

Class imbalance analysis results.

Dataset	Model	Acc.	Prec.	Recall	F1	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
Baseline	LR	0.9838	0.9713	0.9062	0.9376	0.9877	0.9820	0.0044	0.9760	0.9881
Baseline	DT	0.9731	0.9124	0.8839	0.8980	0.9354	0.9690	0.0021	0.9661	0.9718
Baseline	KNN	0.9659	0.9326	0.8036	0.8633	0.9316	0.9623	0.0029	0.9583	0.9663
Baseline	SVM	0.9844	0.9670	0.9152	0.9404	0.9831	0.9820	0.0039	0.9766	0.9875
Baseline	NC	0.9575	0.8964	0.7723	0.8297	0.9955	0.9487	0.0049	0.9419	0.9555
SMOTE	LR	0.9767	0.9004	0.9286	0.9143	0.9710	0.9753	0.0026	0.9717	0.9789
SMOTE	DT	0.8654	0.4987	0.8482	0.6281	0.8640	0.9224	0.0065	0.9133	0.9315
SMOTE	KNN	0.5356	0.2223	0.9866	0.3629	0.8262	0.7121	0.0159	0.6900	0.7342
SMOTE	SVM	0.9533	0.7829	0.9018	0.8382	0.9642	0.9733	0.0037	0.9681	0.9785
SMOTE	NC	0.9569	0.8958	0.7679	0.8269	0.9688	0.8415	0.0096	0.8281	0.8549
Undersampling	LR	0.9743	0.8787	0.9375	0.9071	0.9851	0.9560	0.0083	0.9445	0.9675
Undersampling	DT	0.9330	0.6818	0.9375	0.7895	0.9349	0.9187	0.0129	0.9008	0.9366
Undersampling	KNN	0.9569	0.8220	0.8661	0.8435	0.9409	0.9092	0.0185	0.8835	0.9348
Undersampling	SVM	0.9665	0.8360	0.9330	0.8819	0.9831	0.9618	0.0043	0.9558	0.9677
Undersampling	NC	0.9575	0.8964	0.7723	0.8297	0.9858	0.8623	0.0176	0.8379	0.8868

Table 7

Baseline — BoW.

Model	Acc.	Prec.	Recall	F1	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
LR	0.9838	0.9713	0.9062	0.9376	0.9877	0.9820	0.0044	0.9760	0.9881
DT	0.9731	0.9124	0.8839	0.8980	0.9354	0.9690	0.0021	0.9661	0.9718
KNN	0.9659	0.9326	0.8036	0.8633	0.9316	0.9623	0.0029	0.9583	0.9663
SVM	0.9844	0.9670	0.9152	0.9404	0.9831	0.9820	0.0039	0.9766	0.9875
NC	0.9575	0.8964	0.7723	0.8297	0.9955	0.9487	0.0049	0.9419	0.9555

Word2Vec), yielding a total of $5 \times 8 \times 3 = 120$ combinations.

To maintain conciseness, Tables 11–13 reports only the top-three combinations per vectorizer ranked by F1-score. The top-5 feature selection combinations for each vectorizer, ranked by F1-score. For BoW, LASSO dominates all three positions, paired with XGB, SVM, and RF respectively, all achieving identical accuracy (0.9820) with marginal F1 differences (0.9296–0.9306).

This consistency indicates that the L1-regularized feature selection is well-suited to the sparse BoW representation, effectively identifying the most discriminative subset of tokens regardless of the downstream classifier. For TF-IDF, the best results are achieved by MI+GB (F1: 0.9374) and LASSO+GB (F1: 0.9371), confirming that Gradient Boosting benefits particularly from the weighted term representations produced by TF-IDF.

The third-ranked combination (RFE+XGB, F1: 0.9349) further underscores the strong performance of ensemble classifiers with TF-IDF features under feature selection. For Word2Vec, LASSO+SVM yields the highest F1 (0.9284), followed by RFE+XGB (0.9220) and LASSO+XGB (0.9206). Overall, TF-IDF achieves the highest F1-scores across all three vectorizers under feature selection, while BoW produces more consistent results across classifier choices.

LASSO emerges as the most frequently appearing feature selection method across all three vectorizers, suggesting that L1 regularization provides a reliable and computationally efficient selection strategy for SMS spam detection regardless of the feature representation used.

Scaling analysis

Scaling experiments were conducted across eight strategies, three vectorizers, and five classifiers, yielding a total of $8 \times 3 \times 5 = 120$ combinations. Scaling is applied exclusively to the training set and then transformed on the test set to prevent data leakage. Tables 14–16 presents the top-5 combinations (f1-score) per vectorizer; the complete results are provided in the supplementary material.

Ensemble learning analysis

XGB outperforms GB and RF across all vectorizers (BoW F1: 0.937,

Table 8

Baseline — TF-IDF.

Model	Acc.	Prec.	Recall	F1	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
LR	0.9707	0.9487	0.8259	0.8831	0.9887	0.9685	0.0025	0.9650	0.9719
DT	0.9731	0.8809	0.9241	0.9020	0.9524	0.9643	0.0068	0.9550	0.9737
KNN	0.9767	0.9426	0.8795	0.9099	0.9759	0.9779	0.0070	0.9682	0.9877
SVM	0.9826	0.9535	0.9152	0.9339	0.9903	0.9828	0.0033	0.9782	0.9874
NC	0.9533	0.8349	0.8125	0.8235	0.9950	0.9561	0.0065	0.9471	0.9652

Table 9

Baseline — Word2Vec.

Model	Acc.	Prec.	Recall	F1	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
LR	0.9749	0.9550	0.8527	0.9009	0.9897	0.9713	0.0062	0.9627	0.9799
DT	0.9348	0.7511	0.7679	0.7594	0.8642	0.9405	0.0101	0.9265	0.9545
KNN	0.9677	0.8602	0.9062	0.8826	0.9706	0.9723	0.0076	0.9617	0.9829
SVM	0.9791	0.9655	0.8750	0.9180	0.9928	0.9805	0.0032	0.9761	0.9849
NC	0.9587	0.8088	0.9062	0.8547	0.9740	0.9420	0.0087	0.9300	0.9541

Table 10

Hyperparameter optimization results.

Algorithm	Vectorizer	Before	After	Key Parameters
LR	BoW	0.9838	0.9861	C= 100, ngram= (1,2)
LR	TF-IDF	0.9707	0.9850	C= 10, ngram= (1,2), max_features= 10000
DT	BoW	0.9731	0.9743	max_depth=None, ngram= (1,2)
DT	TF-IDF	0.9731	0.9743	max_depth=None, ngram= (1,2)
KNN	BoW	0.9659	0.9450	n_neighbors= 3, weights=distance
KNN	TF-IDF	0.9767	0.9492	n_neighbors= 3, weights=distance
SVM	BoW	0.9844	0.9844	C= 10, kernel=linear, ngram= (1,2)
SVM	TF-IDF	0.9826	0.9868	C= 1, kernel=linear, ngram= (1,2)
NC	BoW	0.9575	0.9581	shrink_threshold=None, ngram= (1,2)
NC	TF-IDF	0.9533	0.9557	shrink_threshold=None, ngram= (1,2)

TF-IDF: 0.928, W2V: 0.925) while being far faster than GB. RF achieves Precision= 1.000 at the cost of lower Recall (conservative strategy). BoW > TF-IDF > Word2Vec ordering is preserved in all ensemble configurations (Table 17).

Table 11

Feature selection results — BoW (top5 combinations).

FS Method	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
LASSO	XGB	0.9820	0.9663	0.8973	0.9306	0.9835	0.9826	0.0032	0.9781	0.9870
LASSO	SVM	0.9820	0.9755	0.8884	0.9299	0.9786	0.9854	0.0037	0.9803	0.9905
LASSO	RF	0.9820	0.9802	0.8839	0.9296	0.9865	0.9836	0.0036	0.9786	0.9886
Chi2	XGB	0.9814	0.9573	0.9018	0.9287	0.9848	0.9774	0.0027	0.9737	0.9811
MI	XGB	0.9814	0.9662	0.8929	0.9281	0.9852	0.9777	0.0027	0.9740	0.9814

Table 12

Feature selection results — TF-IDF (top5 combinations).

FS Method	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
MI	GB	0.9838	0.9758	0.9018	0.9374	0.9826	0.9738	0.0024	0.9705	0.9771
LASSO	GB	0.9838	0.9805	0.8973	0.9371	0.9841	0.9767	0.0053	0.9692	0.9841
RFE	XGB	0.9832	0.9757	0.8973	0.9349	0.9785	0.9790	0.0041	0.9733	0.9847
RFI	XGB	0.9832	0.9851	0.8884	0.9343	0.9828	0.9761	0.0048	0.9695	0.9828
RFI	RF	0.9832	0.9900	0.8839	0.9340	0.9908	0.9813	0.0026	0.9776	0.9849

The incorrectly classified sample analysis

Despite the integrated studies, there were still instances that were incorrectly classified. The false negative (FN) cases where spam messages are incorrectly classified as legitimate (ham) represent the most critical source of error in spam detection systems.

Unlike false positives, which merely inconvenience users by misclassifying harmless messages, false negatives directly compromise the security and reliability of the system by allowing malicious or fraudulent content to reach recipients. A single false negative can lead to exposure to phishing links, financial scams, or adult content, thereby reducing user trust in automated filtering mechanisms (Table 18).

The best model (BoW + QuantileTransformer + LR, Acc: 0.989, F1: 0.956) produced 18 FN and 1 FP from 1672 test instances. False negatives fall into six categories: personal/chat spam (6), humor/joke spam (2), short/noisy (3), service/subscription (4), corporate language (1), and financial fraud (2). The single FP is a legitimate Airtel broadband confirmation (spam prob: 0.615).

These findings highlight the structural limitations of token-frequency methods in capturing semantic intent and evolving spam tactics.

Statistical significance analysis

To assess whether performance differences among the top-performing configurations are statistically meaningful, the Wilcoxon signed-rank test was applied to the 5-fold cross-validation accuracy scores of the three best models. The results indicate no statistically

Table 13

Feature selection results — Word2Vec (top5 combinations).

FS Method	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
LASSO	SVM	0.9814	0.9617	0.8973	0.9284	0.9890	0.9800	0.0042	0.9742	0.9858
RFE	XGB	0.9803	0.9799	0.8705	0.9220	0.9899	0.9787	0.0024	0.9754	0.9820
LASSO	XGB	0.9797	0.9657	0.8795	0.9206	0.9882	0.9810	0.0036	0.9760	0.9860
Chi2	XGB	0.9785	0.9519	0.8839	0.9167	0.9832	0.9777	0.0050	0.9707	0.9847
MI	XGB	0.9773	0.9471	0.8795	0.9120	0.9855	0.9792	0.0046	0.9728	0.9856

Table 14

Top5 scaling combinations — BoW vectorization.

Scaler	Model	Acc.	Prec.	Recall	F1	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
QuantileTransf.	LR	0.9886	0.9952	0.9196	0.9559	0.9905	0.9869	0.0019	0.9843	0.9895
Binarizer	LR	0.9874	0.9951	0.9107	0.9510	0.9900	0.9861	0.0027	0.9823	0.9900
Binarizer	SVM	0.9862	0.9855	0.9107	0.9466	0.9901	0.9844	0.0022	0.9813	0.9874
RobustScaler	SVM	0.9844	0.9670	0.9152	0.9404	0.9831	0.9820	0.0039	0.9766	0.9875
RobustScaler	LR	0.9838	0.9713	0.9062	0.9376	0.9877	0.9820	0.0044	0.9760	0.9881

Table 15

Top5 scaling combinations — TF-IDF vectorization.

Scaler	Model	Acc.	Prec.	Recall	F1	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
Normalizer	SVM	0.9850	0.9807	0.9062	0.9420	0.9934	0.9810	0.0072	0.9710	0.9911
RobustScaler	SVM	0.9844	0.9760	0.9062	0.9398	0.9934	0.9808	0.0073	0.9706	0.9910
QuantileTransf.	SVM	0.9827	0.9803	0.8884	0.9321	0.9891	0.9836	0.0055	0.9760	0.9912
MinMaxScaler	SVM	0.9827	0.9710	0.8973	0.9327	0.9903	0.9808	0.0053	0.9734	0.9882
Binarizer	SVM	0.9803	0.9704	0.8795	0.9227	0.9866	0.9838	0.0060	0.9755	0.9922

Table 16

Top5 scaling combinations — Word2Vec vectorization.

Scaler	Model	Acc.	Prec.	Recall	F1	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
Normalizer	SVM	0.9808	0.9706	0.8839	0.9252	0.9927	0.9779	0.0041	0.9744	0.9815
MinMaxScaler	LR	0.9797	0.9612	0.8839	0.9209	0.9915	0.9838	0.0044	0.9800	0.9877
MaxAbsScaler	LR	0.9797	0.9612	0.8839	0.9209	0.9915	0.9838	0.0044	0.9800	0.9877
MinMaxScaler	SVM	0.9767	0.9343	0.8884	0.9108	0.9908	0.9823	0.0055	0.9775	0.9871
RobustScaler	KNN	0.9713	0.8793	0.9107	0.8947	0.9743	0.9726	0.0092	0.9645	0.9806

Table 17

Ensemble learning results.

Vectorizer	Model	Acc.	Prec.	Recall	F1	ROC-AUC	CV-Acc	CV-Std	CI-95% Low	CI-95% High
BoW	GB	0.9755	0.9463	0.8661	0.9044	0.9888	0.9769	0.0054	0.9694	0.9844
BoW	XGB	0.9838	0.9758	0.9018	0.9374	0.9839	0.9820	0.0039	0.9766	0.9874
BoW	RF	0.9761	1.0000	0.8214	0.9020	0.9949	0.9782	0.0035	0.9733	0.9831
TF-IDF	GB	0.9797	0.9703	0.8750	0.9202	0.9861	0.9749	0.0054	0.9673	0.9824
TF-IDF	XGB	0.9814	0.9617	0.8973	0.9284	0.9798	0.9800	0.0046	0.9736	0.9864
TF-IDF	RF	0.9773	1.0000	0.8304	0.9073	0.9965	0.9767	0.0032	0.9722	0.9811
Word2Vec	GB	0.9761	0.9510	0.8661	0.9065	0.9856	0.9744	0.0064	0.9654	0.9833
Word2Vec	XGB	0.9808	0.9706	0.8839	0.9252	0.9868	0.9779	0.0025	0.9745	0.9814
Word2Vec	RF	0.9653	0.9882	0.7500	0.8528	0.9810	0.9654	0.0051	0.9582	0.9725

significant difference between BoW + QuantileTransformer + LR and BoW + Binarizer + LR (stat=2.5, $p = 0.500$), nor between BoW + QuantileTransformer + LR and BoW + Binarizer + SVM (stat=0.0, $p = 0.125$), nor between BoW + Binarizer + LR and BoW + Binarizer + SVM (stat=1.0, $p = 0.125$). All p -values exceed the 0.05 significance threshold, indicating that the three configurations are statistically equivalent in terms of cross-validation accuracy. Consequently, the selection among these top configurations may be guided by secondary criteria such as computational cost and precision-recall trade-off rather than raw accuracy alone. Among the three, BoW + QuantileTransformer + LR is reported as the primary best configuration due to its highest mean CV-accuracy (0.9869) and F1-score (0.9559).

Comparison with previous studies

Table 19 presents the methodologies and performance results used in previous studies. This comparative overview highlights the similarities and differences between our proposed approach and previous studies. The proposed pipeline achieves Accuracy 0.989 and F1 0.956, competitive with LLM-based approaches (0.989–0.991) at substantially lower computational cost. Unlike prior work, this study provides a comprehensive comparison across 8 classifiers, 3 vectorizers, 5 feature selection methods, and 8 scaling strategies within a single reproducible framework.

Among recent approaches, Shen et al. [22] proposed BERT-G3CN, integrating BERT embeddings with triple-graph convolutional

Table 18

False negative instances — (BoW + QuantileTransformer + LR).

#	Original SMS (truncated)	Spam Prob	Category
1	Would you like to see my XXX pics they are so hot...	0.0000	Personal/Chat Spam
2	Hello darling how are you today? I would love to have a chat...	0.0000	Personal/Chat Spam
3	Latest News! Police station toilet stolen, cops have nothing...	0.0001	Humor/Joke Spam
4	Did you hear about the new Divorce Barbie? It comes with all...	0.0004	Humor/Joke Spam
5	Oh my god! I've found your number again! Text me back...	0.0014	Personal/Chat Spam
6	Babe: U want me dont u baby! Im nasty and have a thing 4...	0.0020	Personal/Chat Spam
7	ringtoneking 84484	0.0075	Short/Noisy
8	You will be receiving this weeks Triple Echo ringtone shortly.	0.0108	Service/Subscription
9	2/2 146tf150p	0.0131	Short/Noisy
10	Bank of granite issues strong-buy explosive pick for our members.	0.0196	Financial Fraud
11	Dorothy@kiefier.com Bank of Granite issues Strong-Buy...	0.0196	Financial Fraud
12	SMS. ac sun0819 posts HELLO: You seem cool	0.0282	Short/Noisy
13	Sorry I missed your call. I'm on 07090201529	0.0425	Service/Subscription
14	FreeMsg: Feelin kinda lnly hope u like 2 keep me company!...	0.0574	Personal/Chat Spam
15	You have important customer service announcement from premier.	0.0725	Corporate Language
16	Missed call alert. These numbers called but left no message.	0.1388	Service/Subscription
17	Sorry! U can not unsubscribe yet. THE MOB offer package...	0.2610	Service/Subscription
18	Hi babe its Chloe, how r u? I was smashed on saturday night...	0.3383	Personal/Chat Spam

networks and achieving 99.28% accuracy on the UCI benchmark the highest reported result to date. However, the authors do not report the train/test split ratio, cross-validation strategy, or data leakage prevention measures, limiting the reproducibility and direct comparability of their results. The proposed pipeline achieves competitive accuracy (98.86%) while surpassing BERT-G3CN in spam-class precision (0.9952 vs. 0.9500), indicating fewer false alarms. Furthermore, BERT-G3CN requires approximately 305 s of training time and up to 6.28 GB of GPU memory, whereas the best configuration of the proposed pipeline completes training in under 2 s on a standard CPU without GPU dependency a difference of two orders of magnitude that makes the classical ML approach substantially more practical for resource-constrained deployments.

Limitations

This study has several limitations that should be considered when

interpreting the results. First, all experiments are conducted on a single benchmark dataset, which consists of English-language messages collected primarily from UK sources in the early 2010s. The dataset may not reflect the linguistic diversity, evolving spam tactics, or multilingual characteristics of contemporary SMS traffic. Second, the analysis of incorrectly classified samples reveals that token-frequency methods are fundamentally limited in their ability to detect contextually disguised spam such as personal/chat-mimicking messages or humor-format spam where the intent cannot be inferred from word frequencies alone. Third, Word2Vec embeddings (Google news) showed limited benefit over BoW/TF-IDF, possibly due to domain mismatch between the training corpus and SMS language. Fourth, the study does not evaluate real-time or online learning scenarios, which may be relevant in dynamic spam environments where spam patterns change over time.

Conclusions

This study proposed and evaluated an integrated machine learning pipeline for SMS spam detection, combining multiple feature extraction methods, feature selection strategies, scaling approaches, ensemble learning, and hyperparameter optimization within a single reproducible framework. The pipeline was evaluated on the UCI SMS Spam Collection v.1 dataset using eight classifiers (LR, DT, KNN, SVM, NC, GB, XGB, RF), three vectorizers (BoW, TF-IDF, Word2Vec), five feature selection methods (Chi², MI, LASSO, RFI, RFE), and eight scaling strategies across systematic experiments.

The experimental results demonstrate that feature extraction method selection has a substantial impact on classification performance. BoW consistently outperforms TF-IDF and Word2Vec across most classifiers, as its simple term frequency representation effectively captures the lexical patterns that distinguish spam from ham in the UCI dataset. Text preprocessing contributes positively to classification performance by preserving numerical content as a discriminative signal while reducing vocabulary size.

Class imbalance analysis reveals that neither SMOTE nor random undersampling improves upon the unbalanced baseline. Both resampling strategies degrade classification performance, with SMOTE causing a particularly severe drop for KNN (F1: 0.863 → 0.363). This finding confirms that BoW features inherently provide sufficient class separation for this dataset, and that correction for class imbalance is not necessary when frequency-based representations are used.

Feature selection consistently improves performance over the unselected baseline across most classifier-vectorizer combinations. LASSO emerges as the most reliable selection method, dominating the top-three combinations for both BoW and Word2Vec. Filter-based methods such as Chi-square and Mutual Information achieve results comparable to the more computationally expensive wrapper-based RFE, making them preferable in practice. Among ensemble methods, XGBoost achieves the best F1-scores across all three vectorizers (BoW: 0.937, TF-IDF: 0.928, Word2Vec: 0.925) while offering substantially lower training time than Gradient Boosting. Hyperparameter optimization via GridSearchCV

Table 19

A comparison with previous best studies.

Study	Method	Feature Extraction	Feature Selection	HPO	Ensemble	Best Acc.
Tekerek et al. [7]	SVM	TF-IDF	No	Yes	No	0.983
Gupta et al. [8]	NB, SVM, DT, RF	TF-IDF	Yes	No	RF	0.980
Sethi et al. [9]	NB, DT, RF	BoW	No	No	No	0.975
Pudasaini et al. [10]	RVM	TF-IDF	No	Yes	No	0.978
Sjarif et al. [11]	RF	TF-IDF	No	No	RF	0.979
Roy et al. [17]	LSTM+GRU	Embeddings	No	Yes	Yes	0.986
Basumatary et al. [13]	LR, SVM, RF, XGB	TF-IDF	No	No	RF	0.982
Salman et al. [23]	GPT-4, LLaMA-2	LLM	No	Yes	Yes	0.991
Uddin et al. [24]	RoBERTa+XAI	LLM	No	Yes	No	0.989
Shen et al. [22]	BERT- G3CN	BERT, Graph	No	Yes	No	0.993
This study	LR, DT, KNN, SVM, NC	BoW, TF-IDF, Word2Vec	Chi2, MI, LASSO, RFI, RFE	Yes	GB, XGB, RF	0.989

further improves performance for all classifiers, with the most notable gains observed for SVM and the adoption of bigram features.

The best overall configuration BoW combined with QuantileTransformer scaling and Logistic Regression achieves an accuracy of 0.989, precision of 0.995, recall of 0.920, F1-score of 0.956, and ROC-AUC of 0.990. Of 1672 test instances, only 18 spam messages were misclassified as ham and 1 ham message as spam. Analysis of false negatives reveals that the primary challenge categories for token-frequency methods are personal and chat-mimicking spam, humor-format spam, and corporate-language spam message types that require semantic intent recognition beyond what BoW representations can provide.

Several directions emerge from the findings and limitations of this study. First, evaluation on larger, more recent, and multilingual SMS datasets would provide stronger evidence for the generalizability of the proposed pipeline. Second, contextual language models such as BERT and RoBERTa could be integrated to address the limitations of token-frequency methods in detecting semantically disguised spam. Third, SMS-specific embeddings trained on contemporary corpora could replace general-purpose Word2Vec/GloVe models. Fourth, sender metadata features (such as phone number patterns, message frequency, and temporal patterns) could complement textual features to enhance detection of service/subscription spam. Fifth, adversarial robustness evaluation against evasion attacks would be a valuable extension, given the known vulnerability of classical ML models to adversarial text modifications.

In summary, this study demonstrates that carefully engineered classical machine learning pipelines remain competitive with deep learning and LLM-based approaches for SMS spam detection, offering a reproducible, interpretable, and computationally efficient framework suitable for resource-constrained deployment scenarios.

CRedit authorship contribution statement

Hidayet TAKCI: Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Tuğçe ŞENOL:** Writing – original draft, Validation, Software, Methodology, Investigation. **Seker Abdulkadir:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology.

Data availability

The dataset is the UCI SMS Spam Collection v.1, available at the UCI Machine Learning Repository [26]. The source code, experimental notebooks, and preprocessing pipeline developed in this study are openly available on GitHub at [kadirseker00/sms_spam](https://github.com/kadirseker00/sms_spam). All experiments are fully reproducible using the provided code and the referenced dataset.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This study is an extended version of the Master's thesis submitted by Tuğçe Şenol to the Department of Computer Engineering, Sivas Cumhuriyet University, 2025 (YÖK Thesis ID: 967602).

Appendix A. Supporting information

Supplementary data associated with this article can be found in the online version at [doi:10.1016/j.jer.2026.05.020](https://doi.org/10.1016/j.jer.2026.05.020).

References

- [1] J. Mounika, SMS spam detection with multinomial Naive Bayes, *Int. J. Sci. Res. Eng. Manag.* (2024).
- [2] D.A. Oyeyemi and A.K. Ojo, "SMS Spam Detection and Classification to Combat Abuse in Telephone Networks Using Natural Language Processing," arXiv preprint, 2023, [doi: 10.48550/arXiv.2406.06578](https://doi.org/10.48550/arXiv.2406.06578).
- [3] M. Iqbal, M.M. Abid, M. Ahmad, F. Khurshid, Study on the effectiveness of spam detection technologies, *Int. J. Inf. Technol. Comput. Sci.* 8 (1) (2016) 11–21, <https://doi.org/10.5815/ijitcs.2016.01.02>.
- [4] H. Al-Kaabi, A.D. Darroudi, A.K. Jasim, Survey of SMS spam detection techniques: a taxonomy, *AlKadhim J. Comput. Sci.* (2024).
- [5] M. Salman, M. Ikram, M.A. Kaafar, Investigating evasive techniques in SMS spam filtering: a comparative analysis of machine learning models, *IEEE Access* 12 (2024), <https://doi.org/10.1109/ACCESS.2024.3364654>.
- [6] S.J. Delany, M.R. Buckley, D. Greene, SMS spam filtering: methods and data, *Expert Syst. Appl.* 39 (10) (2012) 9899–9908, <https://doi.org/10.1016/j.eswa.2012.02.053>.
- [7] A. Tekerek, Support vector machine based spam SMS detection, *Politekn. Derg.* 22 (3) (2019) 779–784.
- [8] M. Gupta, A.A. Bakliwal, S. Agarwal, P. Mehndiratta, A comparative study of spam SMS detection using machine learning classifiers, 2018 Elev. Int. Conf. Contemp. Comput. (IC3) (2018) 1–7, <https://doi.org/10.1109/IC3.2018.8530562>.
- [9] P. Sethi, V. Bhandari, B. Kohli, SMS spam detection and comparison of various machine learning algorithms, 2017 Int. Conf. Comput. Commun. Technol. smart Nation (IC3TSN) (2017) 28–31.
- [10] S. Pudasaini, A. Shakyia, S.P. Pandey, P. Paudel, S. Ghimire, P. Ale, SMS spam detection using relevance vector machine, *Procedia Comput. Sci.* 230 (2023) 337–346, <https://doi.org/10.1016/j.procs.2023.12.083>.
- [11] N.N.A. Sjarif, N.F.M. Azmi, S. Chuprat, H.M. Sarkan, Y. Yahya, S.M. Sam, SMS spam message detection using term frequency-inverse document frequency and random forest algorithm, *Procedia Comput. Sci.* 161 (2019) 509–515, <https://doi.org/10.1016/j.procs.2019.11.105>.
- [12] D. Suleiman, G. Al-Naymat, SMS spam detection using H2O framework, *Procedia Comput. Sci.* 113 (2017) 154–161, <https://doi.org/10.1016/j.procs.2017.08.332>.
- [13] B. Basumatary, T.K. Norbu, S.A. Kokatnoor, S. Kumar, Machine learning models for SMS spam detection, *Int. Conf. Data Sci. Appl.* (2024) 423–433.
- [14] J. Ma, Y. Zhang, J. Liu, K. Yu, X. Wang, Intelligent SMS spam filtering using topic model, *Int. J. Technol. Hum. Interact. (IJTHI)* 12 (2) (2016) 83–95, <https://doi.org/10.4018/IJTHI.2016040106>.
- [15] N.A. Moubayed, T.P. Breckon, P.H. Matthews, A.S. McGough, SMS spam filtering using probabilistic topic modelling and stacked denoising autoencoder. *Lecture Notes in Computer Science*, Springer, 2016, pp. 423–430, https://doi.org/10.1007/978-3-319-45814-4_33.
- [16] M. Popovac, M. Karanovic, S. Sladojevic, M. Arsenovic, A. Anderla, Convolutional neural network based SMS spam detection, 2018 26th Telecommun. Forum (TELFOR) (2018) 1–4.
- [17] P.K. Roy, J.P. Singh, S. Banerjee, Deep learning to filter SMS Spam, *Future Gener. Comput. Syst.* 102 (2020) 524–533, <https://doi.org/10.1016/j.future.2019.09.001>.
- [18] P. Poomka, W. Pongsena, N. Kerdprasop, K. Kerdprasop, SMS spam detection based on long short-term memory and gated recurrent unit, *Int. J. Future Comput. Commun.* 8 (1) (2019) 11–15, <https://doi.org/10.18178/ijfcc.2019.8.1.533>.
- [19] O. Abayomi-Alli, S. Misra, A. Abayomi-Alli, A deep learning method for automatic SMS spam classification: Performance of learning algorithms on indigenous dataset, *Concurr. Comput.* 34 (17) (2022), <https://doi.org/10.1002/cpe.6865>.
- [20] J. De Goma, J.A. Bravo, S. Prudente, R.F. Rondilla, Detection of SMS spam messages using TF-IDF vectorizer and deep learning models, *Proc. 2024 9th Int. Conf. Intell. Inf. Technol.* (2024) 245–249.
- [21] H.C. Altunay, Z. Albayrak, SMS spam detection system based on deep learning architectures for Turkish and English messages, *Appl. Sci.* 14 (24) (2024) 11804.
- [22] L. Shen, Y. Wang, Z. Li, W. Ma, SMS spam detection using BERT and multi-graph convolutional networks, *Int. J. Intell. Netw.* 6 (2025) 79–88, <https://doi.org/10.1016/j.ijin.2025.06.002>.
- [23] M. Salman, M. Ikram, N. Basta, and M.A. Kaafar, "SpaLLM-Guard: Pairing SMS Spam Detection Using Open-source and Commercial LLMs," arXiv preprint, 2025, [doi: 10.48550/arXiv.2501.00000](https://doi.org/10.48550/arXiv.2501.00000).
- [24] M.A. Uddin, M.N. Islam, L. Maglaras, H. Janicke, and I.H. Sarker, "ExplainableDetector: Exploring Transformer-based Language Modeling Approach for SMS Spam Detection with Explainability Analysis," arXiv preprint, 2024, [doi: 10.48550/arXiv.2403.00000](https://doi.org/10.48550/arXiv.2403.00000).
- [25] M. Ahmadi, M. Khajavi, A. Varmaghani, A. Ala, K. Danesh, D. Javaheri, Leveraging large language models for cybersecurity: enhancing sms spam detection with robust and context-aware text classification, *Cyber-Phys. Syst.* (2025) 1–27.
- [26] T.A. Almeida, J.M.G. Hidalgo, A. Yamakami, Contributions to the study of SMS spam filtering: new collection and results, *Proc. 2011 ACM Symp Doc. Eng. (DocEng'11) Mt. View Calif. USA* (2011) 259–262, <https://doi.org/10.1145/2034691.2034742>.
- [27] M.N.W. Nkongolo, News classification and categorization with smart function sentiment analysis, *Int. J. Intell. Syst.* 2023 (1) (2022), <https://doi.org/10.1155/2022/9431835>.
- [28] A. Priyatno, T. Widiyaningtyas, A systematic literature review: recursive feature elimination algorithms, *JITK (J. Ilmu Pengetah. Dan. Teknol. Komput.)* 9 (2024) 196–207, <https://doi.org/10.30865/mib.v8i1.7036>.

- [29] Y. Zhai, W. Song, X. Liu, L. Liu, X. Zhao, A chi-square statistics based feature selection method in text classification, 2018 IEEE 9th International conference software engineering service science (ICSESS) (2018) 160–163.
- [30] J. Vergara, P. Estevez, A review of feature selection methods based on mutual information, *Neural Comput. Appl.* 24 (2019), <https://doi.org/10.1007/s00521-013-1368-0>.
- [31] R. Muthukrishnan, R. Rohini, LASSO: a feature selection technique in predictive modeling for machine learning, 2016 IEEE Int. Conf. Adv. Comput. Appl. (ICACA) (2016) 18–20.
- [32] D.M. Reif, A.A. Motsinger, B.A. McKinney, J.E. Crowe, J.H. Moore, Feature selection using a random forests classifier for the integrated analysis of multiple data types, 2006 IEEE Symp Comput. Intell. Bioinforma. Comput. Biol. (2006) 1–8.
- [33] C. Bentéjac, A. Csörgő, G. Martínez-Muñoz, A comparative analysis of gradient boosting algorithms, *Artif. Intell. Rev.* 54 (3) (2020) 1937–1967, <https://doi.org/10.1007/s10462-020-09896-5>.